

LiveCoding: Higher-Order Functions (MovieDatabase)

```
import java.util.Collection; //Imports nicht vergessen!  
  
//Dies ist die Klasse, die wir implementieren wollen!  
public class MovieDatabase {  
    private final Collection<Movie> movies;  
  
    MovieDatabase(Collection<Movie> movies) {  
        this.movies = movies;  
    }  
}
```

Aufgabe 2 aus Higher Order Functions:

Wir spielen TestDrivenDesign

In diesem Fall haben wir unseren Test bereits geschrieben.

Schauen wir uns den Test und die gegebenen Klassen einmal an!

Jetzt erstellen wir unsere MovieDatabase-Klasse. Sie bekommt im Konstruktor einfach eine Liste von Filmen übergeben und speichert sie.

```

// Schritt 1: Implementierung von `forEach(Consumer<Movie>
action)`

// Die Signatur nutzt Consumer<Movie>
void forEach(Consumer<Movie> action) {
    // Die Implementierung ist einfach:
    // Wir iterieren über alle unsere Filme...
    for (Movie movie : movies) {
        // ...und rufen für jeden Film die übergebene 'action' auf.
        action.accept(movie);
    }

    // Alternativ:
    // return movies.stream()
    // .forEach(action);
}

```

Als Erstes wollen wir eine eigene forEach-Methode
 Der Testfall in unserem Test will diese Methode so aufrufen:
 movieDatabase.forEach(movie -> ...).

IM CODE ANSEHEN -> Testklasse

Analysieren wir das Lambda: movie -> Es nimmt ein Movie-Objekt und gibt
 void zurück.

Welches Functional Interface brauchen wir also?" (Antwort: *Consumer<Movie>*)

Genau. Also implementieren wir die Methode:

Damit haben wir unsere erste HOF fertig.

```

// Schritt 2: Implementierung von `findByCategory(Predicate<Movie> criteria)`
// Die Signatur nutzt Predicate<Movie> und gibt eine neue Collection zurück
Collection<Movie> findByCategory(Predicate<Movie> criteria) {

    // Wir brauchen eine neue Liste für die Ergebnisse
    Collection<Movie> result = new ArrayList<>();

    // Wir iterieren wieder über alle Filme
    for (Movie movie : movies) {
        // Jetzt kommt der Kern: Wir benutzen das Prädikat! Wir rufen die .test() Methode auf.
        if (criteria.test(movie)) {

            // Wenn das Prädikat 'true' zurückgibt, fügen wir den Film zur Ergebnisliste hinzu.
            result.add(movie);
        }
    }

    // Am Ende geben wir die gefilterte Liste zurück.
    return result;

    // Alternative mit Stream:
    // return movies.stream()
    // .filter(criteria)
    // .toList();
}

```

Schritt 3: Implementierung von findByCategory (eine filter-Methode)

Wir wollen eine flexible Filter-Methode.

Der Testfall sieht so aus: `movieDatabase.findByCategory(movie -> movie.hasCategory(Category.SCI FI) && ...)`#

IM CODE ANSEHEN

Analysieren wir dieses Lambda: `movie -> ...boolean....` Es nimmt ein Movie und gibt ein boolean zurück. Welches Interface ist das?" (Antwort: *Predicate<Movie>*)

Und die Methode soll laut Test eine `Collection<Movie>` zurückgeben. Fügen wir sie hinzu:

CODEN

Und das ist alles! Wir haben gerade filter von Hand nachgebaut. Unsere `MovieDatabase` ist jetzt super flexibel. Der Aufrufer kann *jede* beliebige Filterlogik als Lambda übergeben, ohne dass wir die `MovieDatabase`-Klasse jemals wieder

ändern müssen.

LiveCoding: Callbacks (BigBrother)

```
class BigBrotherTest {  
    @Test  
    void subscribeAndNotifyOnNameChanged() {  
  
        Person ada = new Person("Ada");  
        Person grace = new Person("Grace");  
  
        BigBrother bigBrother = new BigBrother()  
            .add(ada)  
            .add(grace);  
  
        ada.setName("Lovelace");  
        grace.setName("Trace");  
  
        assertTrue(bigBrother.logContains("Ada changed name to Lovelace"));  
        assertTrue(bigBrother.logContains("Grace changed name to Trace"));  
    }  
}
```

```
@Test  
void notNotifyIfNotSubscribedOnNameChanged() {  
  
    Person ada = new Person("Ada");  
    BigBrother bigBrother = new BigBrother();  
  
    ada.setName("Lovelace");  
  
    assertFalse(bigBrother.logContains("Ada changed name to  
Lovelace"));  
}
```

Das Szenario (Test-Driven Ansatz)

Ziel: Wir schreiben zuerst den Test, um zu verstehen, wie die Klassen interagieren sollen (API Design).

Wir wollen ein Überwachungssystem bauen.

Wir haben eine Person, und immer wenn diese ihren Namen ändert, soll BigBrother das protokollieren.

Wir wollen aber nicht den Code der Person ändern müssen, wenn wir später z. B. auch noch jemand anderen benachrichtigen wollen.

Wir nutzen Callbacks.

Erst einmal Tests anschauen!

1.1 Arrange (Vorbereitung): Erstellen der "Subjects" (Beobachtete Objekte).

Diese Objekte halten später die Liste der Callbacks.

```
Person ada = new Person("Ada");  
Person grace = new Person("Grace");
```

1.2 Arrange (Verknüpfung): Erstellen des "Observer" (Beobachter).

Das Aufrufen von .add() ist der entscheidende Schritt:

Hier wird das Callback-Lambda aus BigBrother in die Listen von 'ada' und 'grace' eingetragen.

Das entspricht dem 'subscribe' im Observer Pattern

```
BigBrother bigBrother = new BigBrother()
    .add(ada)
    .add(grace);
```

2. Act (Ausführung): Der Zustand der Subjects ändert sich.

von setName() wird nun die Liste der Observer durchlaufen und die Callback-Methode (nameChanged) aufgerufen.

```
ada.setName("Lovelace");
grace.setName("Trace");
```

3. Assert (Überprüfung): Validierung der Seiteneffekte.

Wir prüfen, ob der Callback tatsächlich ausgeführt wurde, indem wir in das Logbuch schauen, das vom Callback befüllt wurde.

```
assertTrue(bigBrother.logContains("Ada changed name to Lovelace"));
assertTrue(bigBrother.logContains("Grace changed name to Trace"));
```

Testet das Negativ-Szenario: Ohne Registrierung (Subscription) darf keine Benachrichtigung erfolgen.

Arrange: Erstellung der Objekte ohne Verknüpfung.

```
Person ada = new Person("Ada");
BigBrother bigBrother = new BigBrother();
```

WICHTIG: bigBrother.add(ada) wird NICHT aufgerufen.

Daher wird kein Callback in 'ada' hinterlegt.

Act: Zustandsänderung auslösen.

Da die Liste der Listener in 'ada' leer ist, passiert hier nichts weiter.

```
ada.setName("Lovelace");
```

Sicherstellen, dass BigBrother nichts mitbekommen hat.

Das Log darf den Eintrag nicht enthalten.

```
assertFalse(bigBrother.logContains("Ada changed name to Lovelace"));
```

Nur ansehen: Das Interface

```
@FunctionalInterface // Stellt sicher, dass das Interface genau eine abstrakte Methode hat -> wichtig für Lambdas!  
interface NameChangedCallback {  
  
    /**  
     * Diese Methode wird vom "Beobachteten" (Person) aufgerufen,  
     * sobald eine Änderung stattgefunden hat.  
     * @param oldName Der Zustand vor der Änderung.  
     * @param newName Der neue Zustand nach der Änderung.  
     */  
    void namedChanged(String oldName, String newName);  
}
```

Ziel: Definieren, wie der Callback aussieht. Warum reicht Consumer<String> hier nicht?

Wir wollen wissen: Was war der alte Name und was ist der neue Name? Ein Standard Consumer<String> nimmt nur einen Wert. Wir definieren also einen eigenen 'Vertrag'.

Dieses Interface definiert die Signatur (den Aufbau) unseres Callbacks.

Nochmal: Warum ein eigenes Interface?

Die Standard-Interfaces von Java akzeptieren meist nur ein Argument.

Da wir aber den alten UND den neuen Namen übergeben wollen, benötigen wir eine Methode mit zwei Parametern.


```

class Person {

    // WICHTIG: Hier speichern wir die Callbacks (die Lambdas)
    private final List<NameChangedCallback> listeners = new ArrayList<>();

    private String name;

    Person(String name) {
        this.name = name;
    }

    String name() {
        return name;
    }

    void setName(String name) {
        // TODO
    }

    // Methode zum Registrieren (das "Abo")
    public void subscribeNameChanged(NameChangedCallback listener) {
        this.listeners.add(listener);
    }
}

```

Ziel: Implementierung der Logik, die den Callback "feuert".

Wir müssen die Klasse Person so umbauen, dass sie nicht mehr nur "dumm" Daten speichert, sondern aktiv Bescheid gibt, wenn sich etwas ändert. Sie muss also "gesprächig" werden.

Die Person muss zwei Dinge tun:

1. Sich merken, wer informiert werden will (Liste von Callbacks).
2. Wenn sich der Name ändert, alle in der Liste anrufen.

private final List<NameChangedCallback> listeners = new ArrayList<>();

Das "Telefonbuch": Hier merken wir uns alle, die benachrichtigt werden wollen. Wir nutzen das Interface 'NameChangedCallback', das wir schon vorher definiert haben.

"Die Person muss sich merken, wer an ihr interessiert ist. Dafür brauchen wir eine Liste. Aber eine Liste von was?

Nicht von Strings oder Integers, sondern von unserem neuen Interface

NameChangedCallback.

Das ist quasi unser Verteilerkreis."

```
public void subscribeNameChanged(NameChangedCallback listener) {  
this.listeners.add(listener); }
```

Die "Abo-Funktion": Über diese Methode können sich Fremde (wie BigBrother) in unser Telefonbuch eintragen.

"Jetzt brauchen wir noch eine Öffnung nach außen. Wir erlauben anderen Objekten, sich in diesen Verteilerkreis einzutragen.

Das nennt man 'Subscribing' oder 'Registrieren'. Wichtig ist: Die Klasse Person weiß gar nicht, *wer* sich da einträgt.

Es ist ihr egal, ob es BigBrother oder jemand anderes ist. Sie kennt nur das Interface."

```

class Person {
    ...

    void setName(String newName) {
        // Alten Zustand sichern
        String oldName = this.name;

        // Zustand ändern
        this.name = newName;

        // Alle anrufen!
        listeners.forEach(listener -> listener.namedChanged(oldName, newName));
    }

    ...
}

```

Hier passiert das eigentliche Event.

Wenn sich der Name ändert, müssen wir zwei Dinge tun:

Erstens den Namen tatsächlich ändern. (`this.name = newName;`)

Zweitens – und das ist neu – die Liste durchgehen und 'Bescheid sagen'.

Beachtet hier: Wir übergeben oldName und newName. (alten sichern: `String oldName = this.name;`)

Das ist der Vorteil unseres eigenen Interfaces:

Wir können genau die Daten liefern, die der Beobachter braucht.

`listeners.forEach(listener -> listener.namedChanged(oldName, newName));`

In diesem Schritt lernen wir die **Entkopplung (Decoupling)**.

Ohne Callbacks müsste in setName direkt `bigBrother.log(...)` stehen.

Mit Callbacks (dieser Schritt) ruft Person nur `listener.namedChanged(...)` auf. Die Person kennt die Klasse BigBrother überhaupt nicht.

```

class BigBrother {
    private final List<String> log = new LinkedList<>();

    BigBrother add(Person person) {
        // Hier definieren wir das Verhalten als Lambda!
        // unser Interface: (String, String) -> void
        person.subscribeNameChanged((oldName, newName) -> {
            String message = String.format("%s changed name to %s", oldName, newName);
            log.add(message);
            System.out.println("BigBrother wrote: " + message); // Für Demo-Effekt
        });
        return this;
    }

    boolean logContains(String line) {
        return log.contains(line);
    }
}

```

Ziel: Implementierung der Reaktion. Hier definieren wir das Lambda.

Big Brother:

Er schreibt einfach alles in ein Logbuch. Wir übergeben der Person nun das Lambda, das definiert, was passieren soll.

Zum Lambda: *HIER findet die Eintragung in die Liste von Person statt: BigBrother ruft die Methode der Person auf und übergibt sein Lambda (den Callback).*

Die Signatur: (oldName, newName) ->

Hier passiert das **Type Matching** mit unserem Functional Interface.

Bezug zum Interface: Java erkennt automatisch, dass dieses Lambda zum Interface NameChangedCallback gehört, welches die Methode void namedChanged(String oldName, String newName) definiert.

Type Inference: Wir müssen die Datentypen (String) nicht explizit hinschreiben. Der Compiler "inferiert" (schlussfolgert) sie aus dem Interface.

Parameter: Die Variablennamen oldName und newName sind frei wählbar,

repräsentieren aber die Werte, die später von der Klasse Person (dem Aufrufer) übergeben werden.

Dieses Lambda ist **nicht pure (impure)**, da es den Zustand der Anwendung verändert: Es schreibt in die Liste log. In der Lösung wird explizit kommentiert: `// not pure lambda`. Das ist hier aber gewollt und notwendig, da BigBrother ja genau diesen Zustand (das Logbuch) pflegen soll.

Separation of Concerns: Am Ende fragen: "Musste die Klasse Person wissen, dass es einen BigBrother gibt?" -> Nein, sie kennt nur das Interface NameChangedCallback. Das ist der Gewinn.

Jetzt noch einmal Testcode ansehen!!!